

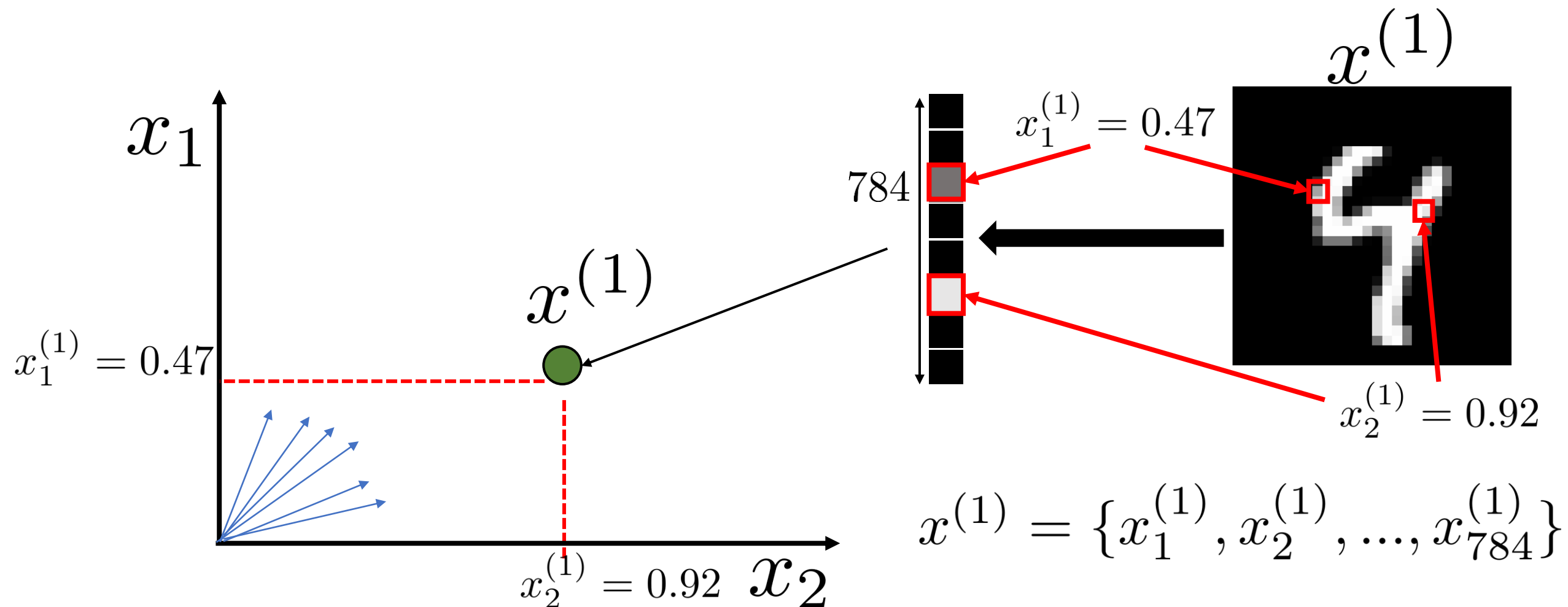
Autoencoders – using Deep Neural Networks for doing unsupervised learning

Slides by Kevin Bryson

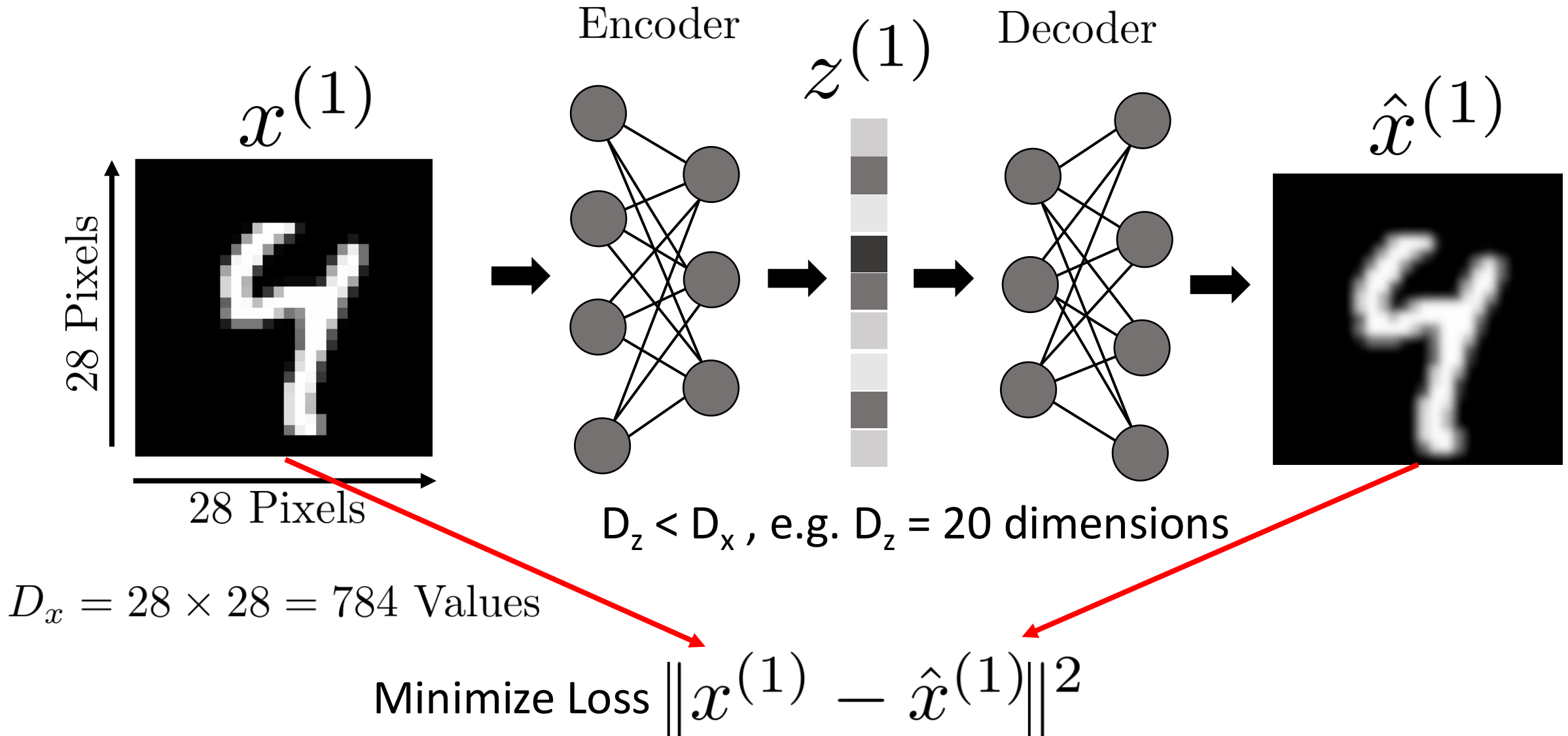
Overview

- We have seen fully connected, CNNs (CovNets), RNNs, Transformer networks.
- These typically do **supervised prediction** (classification or regression).
- But can we use deep neural networks for **unsupervised learning** ?
- Tasks such as **dimensionality reduction / feature extraction** (you should be familiar with PCA for both these applications). Also, **probability density estimation** which could potentially allow **generative models** that can “generate” samples from a probability density.
- In the following lectures we will look at deep learning approaches for unsupervised learning including:
 - **autoencoders (AEs)** – this week
 - **variational autoencoders (VAEs)** and **generative adversarial networks (GANs)** – next week.
- I will first give an intuitive graphical understanding of what these techniques do ... before explaining the mathematical foundation in a bit more detail.

Sample data points in high dimensional spaces



Overview of an autoencoder



Loss Functions

For binary inputs we take the sum of Bernoulli cross-entropies:

$$l(f(x)) = - \sum_k (x_k \log(\hat{x}_k) + (1 - x_k) \log(1 - \hat{x}_k))$$

For real-valued inputs we take the MSE:

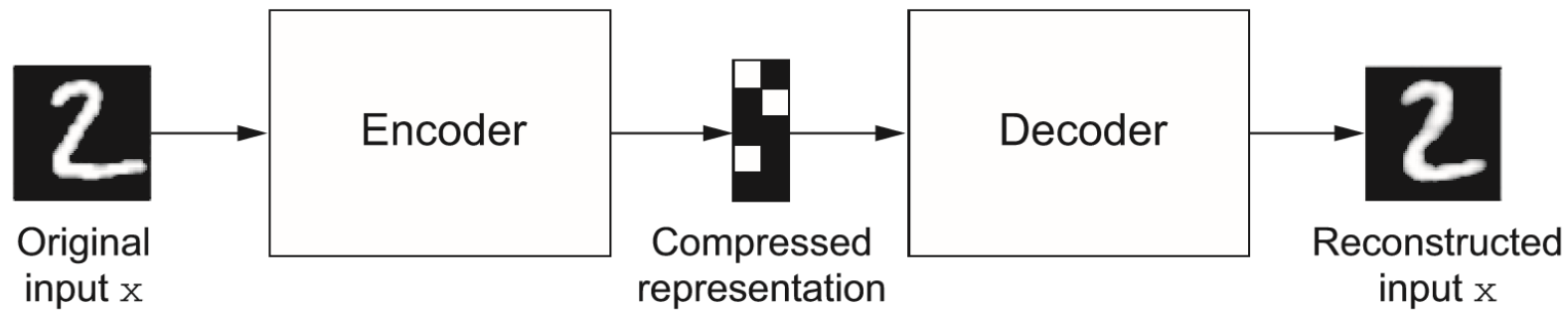
$$l(f(x)) = \frac{1}{2} \sum_k (\hat{x}_k - x)^2$$

(with a linear activation on the output layer).

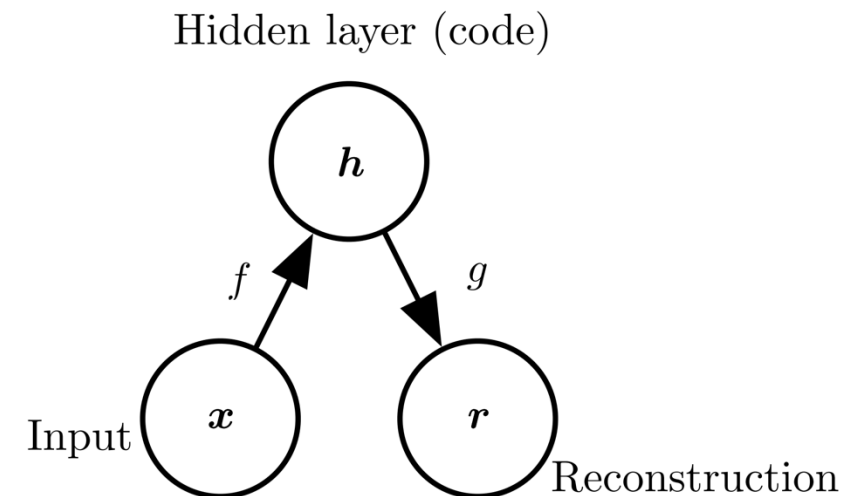
Why is this useful ?

- The 'latent space' z has a lower dimension so this can be seen as a dimensionality reduction technique like PCA. (Essentially it is doing data compression with z being a 'coding' of the original data.)
- The 'latent space' z has most of the important information of the higher dimensional space (otherwise it couldn't reconstruct the images), thus it can be seen as useful for feature extraction like PCA.
- Can be easily trained with lots of unlabelled data so could be used for unsupervised pre-training of the weights of a classification network.
- Is just another fully connected network with a particular loss function so all the previous techniques (gradient back-propagation, SGD, Adam, normalization, etc.) can be used for autoencoders.

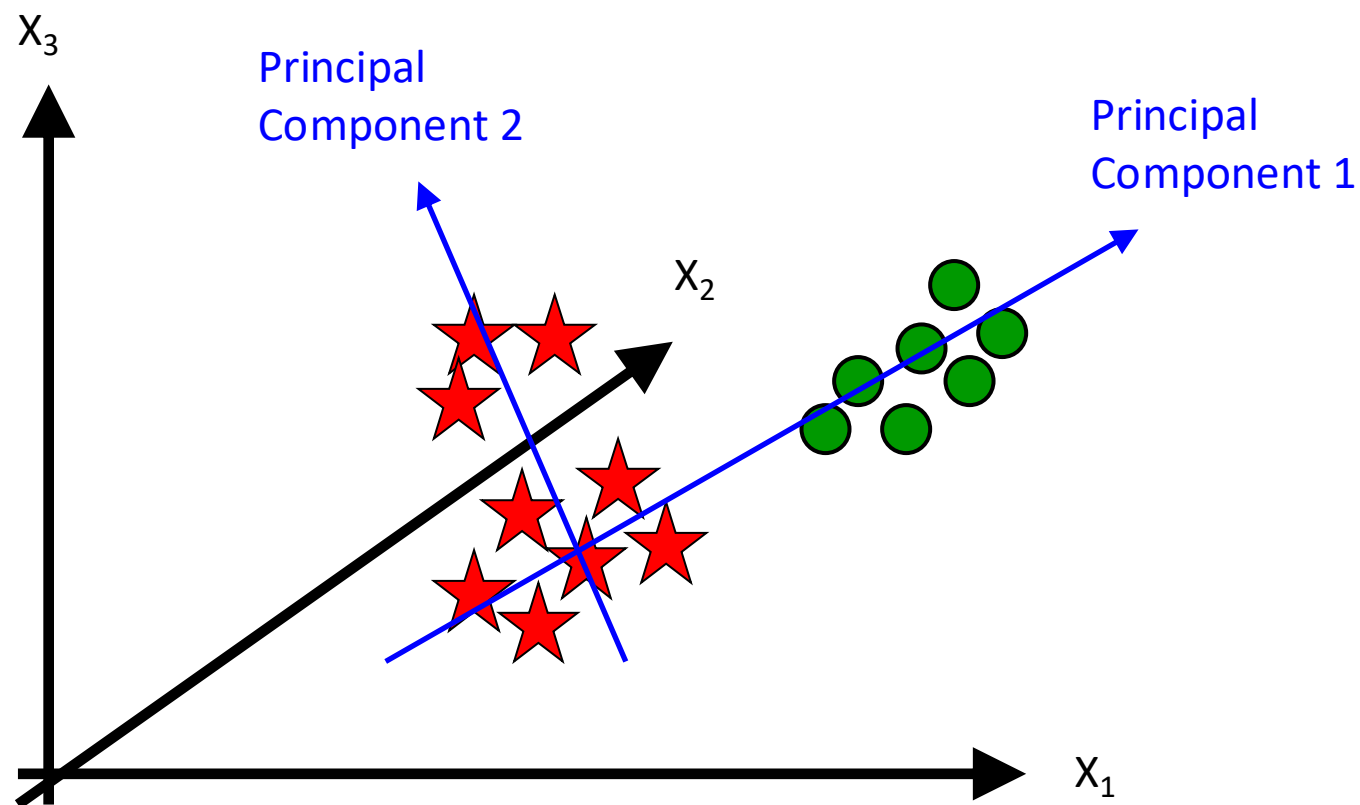
Structure of an autoencoder



- The autoencoder can be seen as having an encoder function f mapping x to h and a decoder function g mapping h back to reconstruction r .

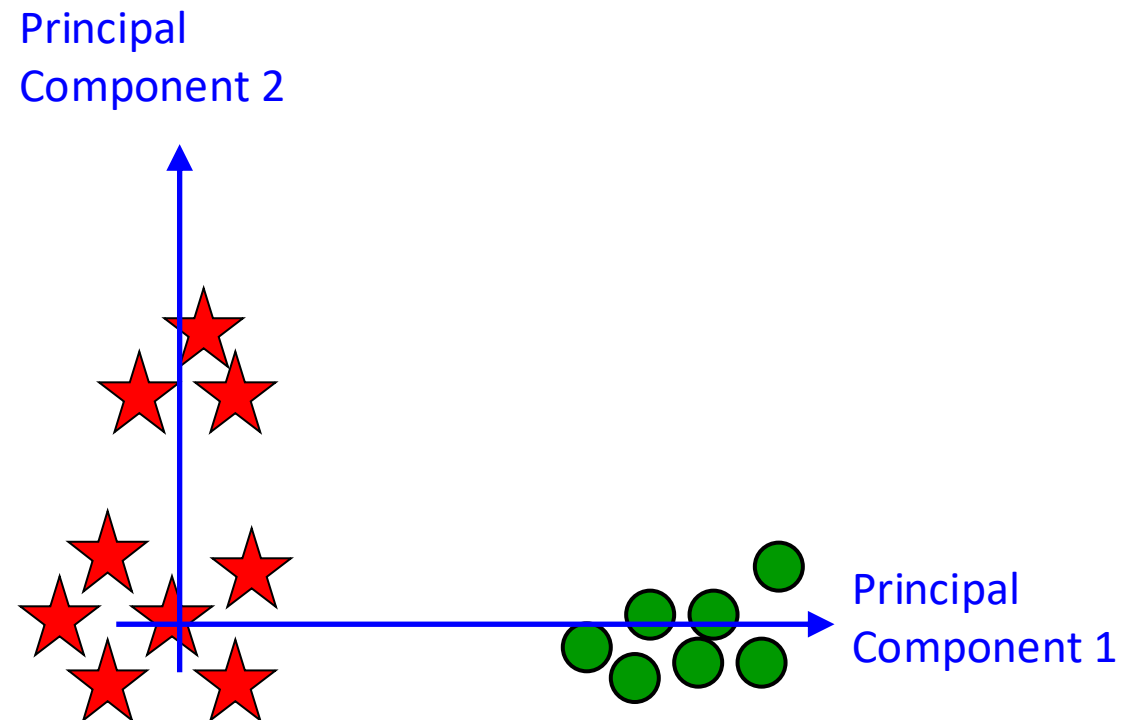


I've mentioned PCA a few times ...



Principal Component Analysis (PCA) is one of the main techniques for dimensionality reduction. It captures as much of the variance in the high dimensional space as is possible in the lower dimension space.

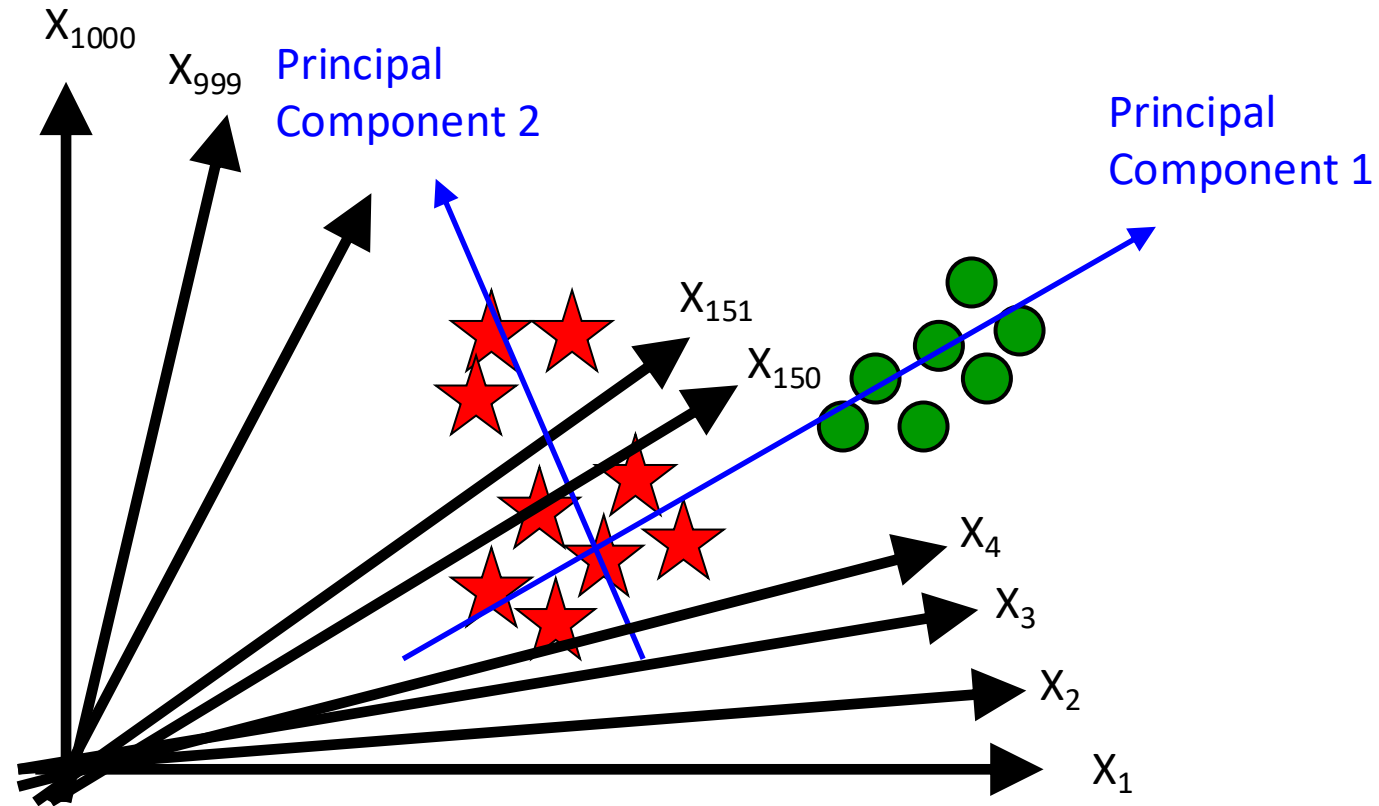
2-D representation of the data



For instance, resulting in a 2-dimensional plot that retains the key variation between samples. These 'principal axes' are defined to be linear combinations of the original gene axes.



More realistic applications of PCA ...



But I could already see this linear pattern in the 3-D plot !

Well it would be much harder to see these trends in a 1000 dimensional space !

But what has this got to do with autoencoders ?

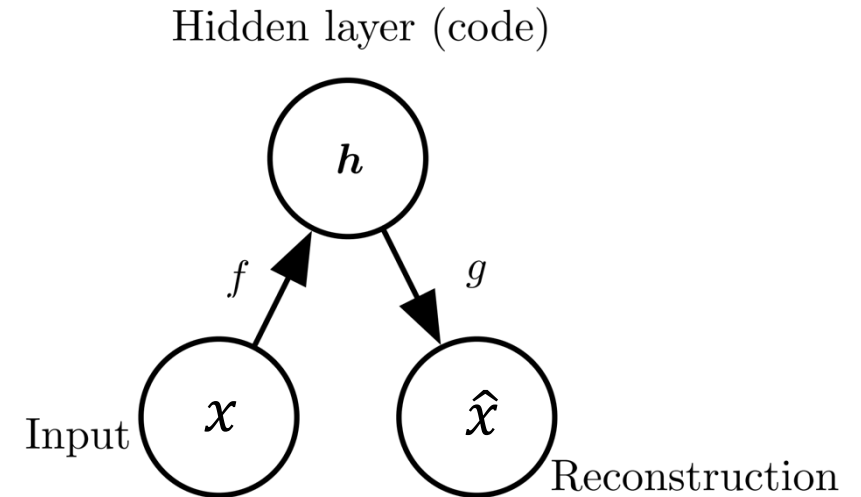
- Well ... if our mapping functions f and g are just linear layers ...
- Then they can be represented by weights W_f and W_g where:

$$h = W_f x$$

$$\hat{x} = W_g h$$

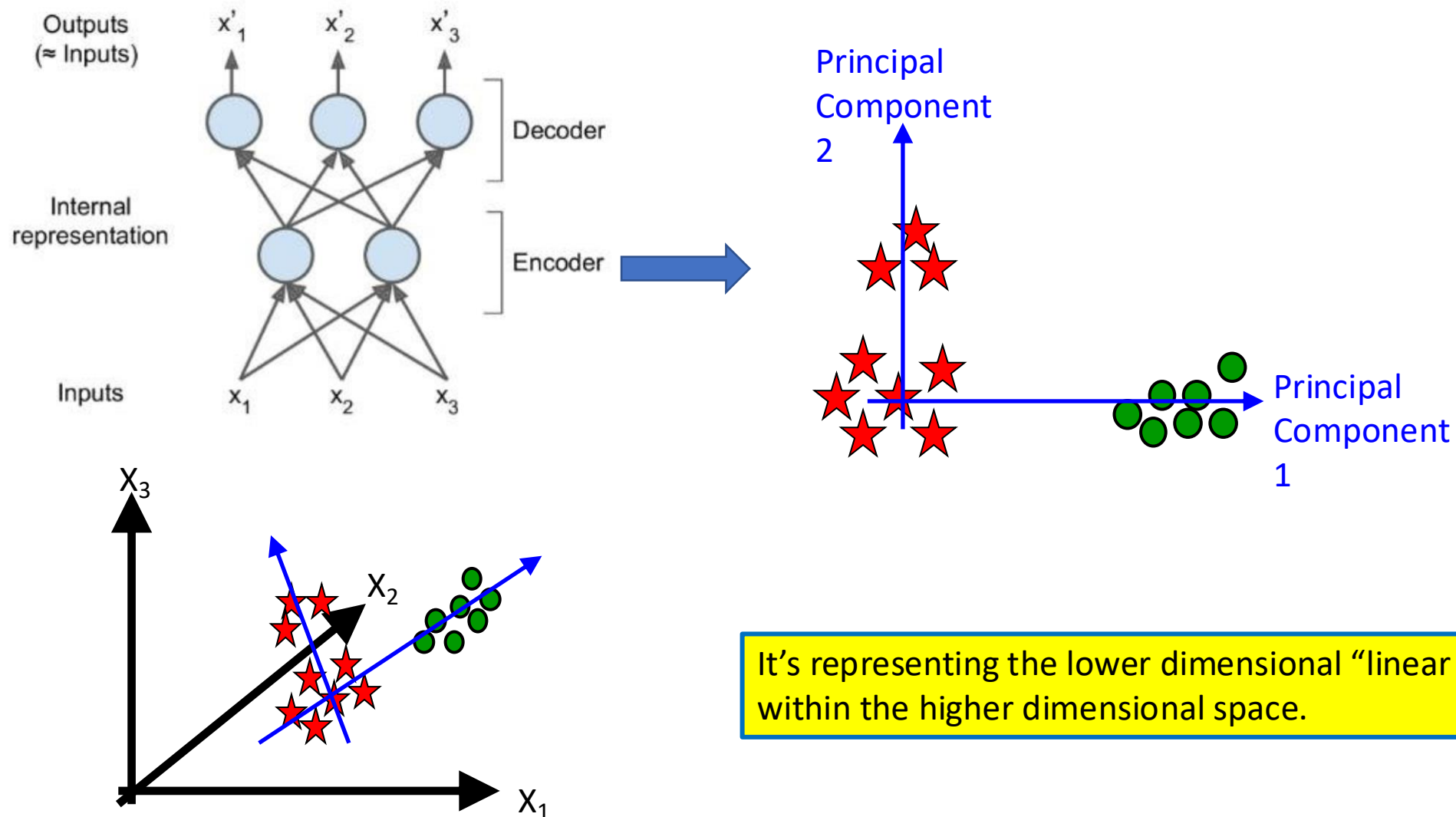
- And we minimize the MSE loss:

$$\|\hat{x} - x\|^2 \text{ or } \|W_g W_f x - x\|^2$$

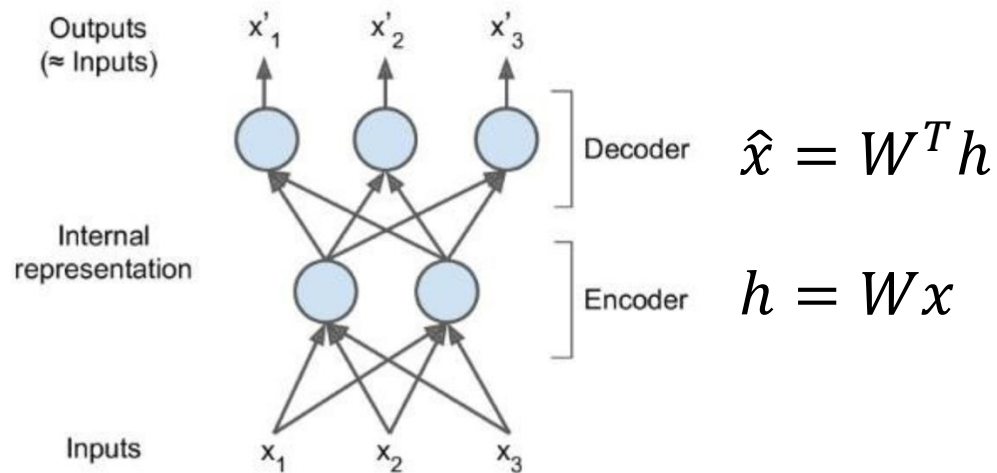


Then we actually get the latent space representation, h , being the principal component (PCA) representation of the x values !

Our linear autoencoder is doing PCA !



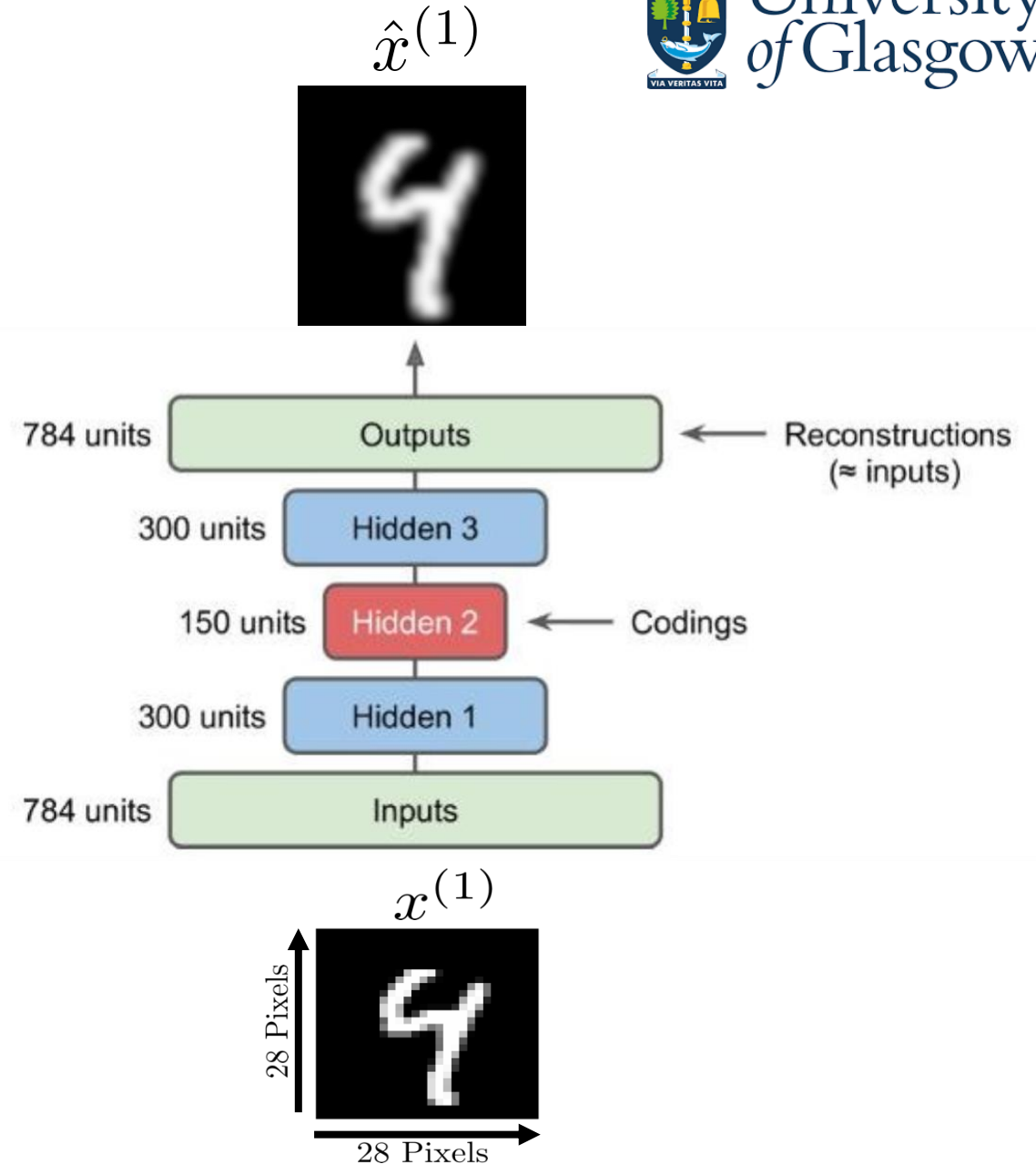
Parameter tying



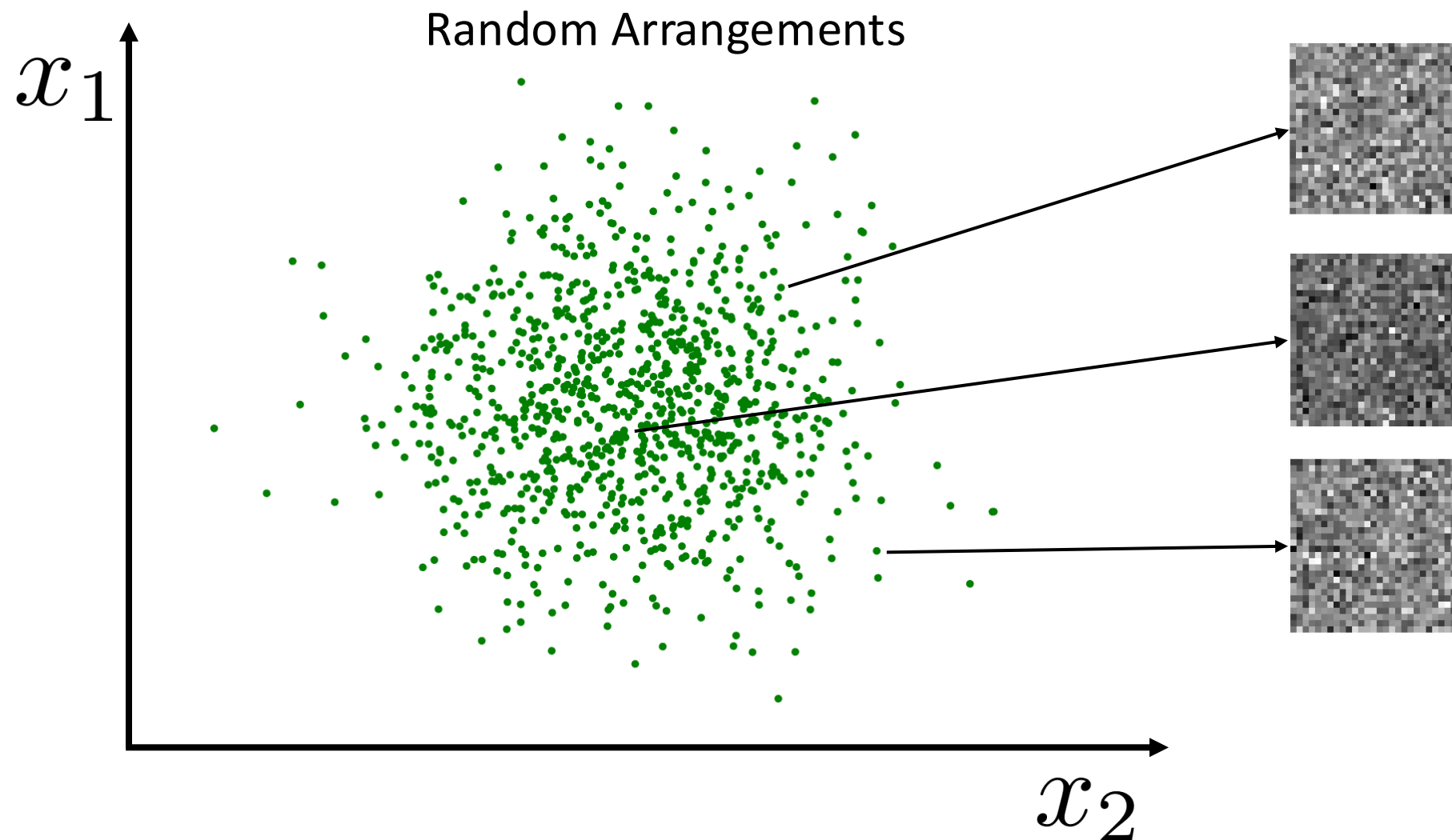
- The decoder is essentially trying to “undo” the encoder’s work.
- Thus, for the linear case, we can use “pseudo inverse” matrices and get the inverse operation using the transpose of W .
- This is another example of parameter or weight tying.
- In this case W would be a 2×3 matrix to map between 3D input column vectors and 2D hidden / latent state.

Non-linear autoencoder

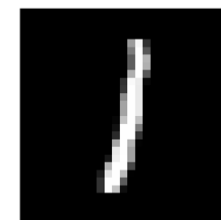
- Instead of just linear layers for the encoding / decoding ... we can use fully connected layers with non-linear activation functions.
- This provides a non-linear mappings between the input space and the latent or hidden space.



Random images have a random arrangement in the data space



Data Set

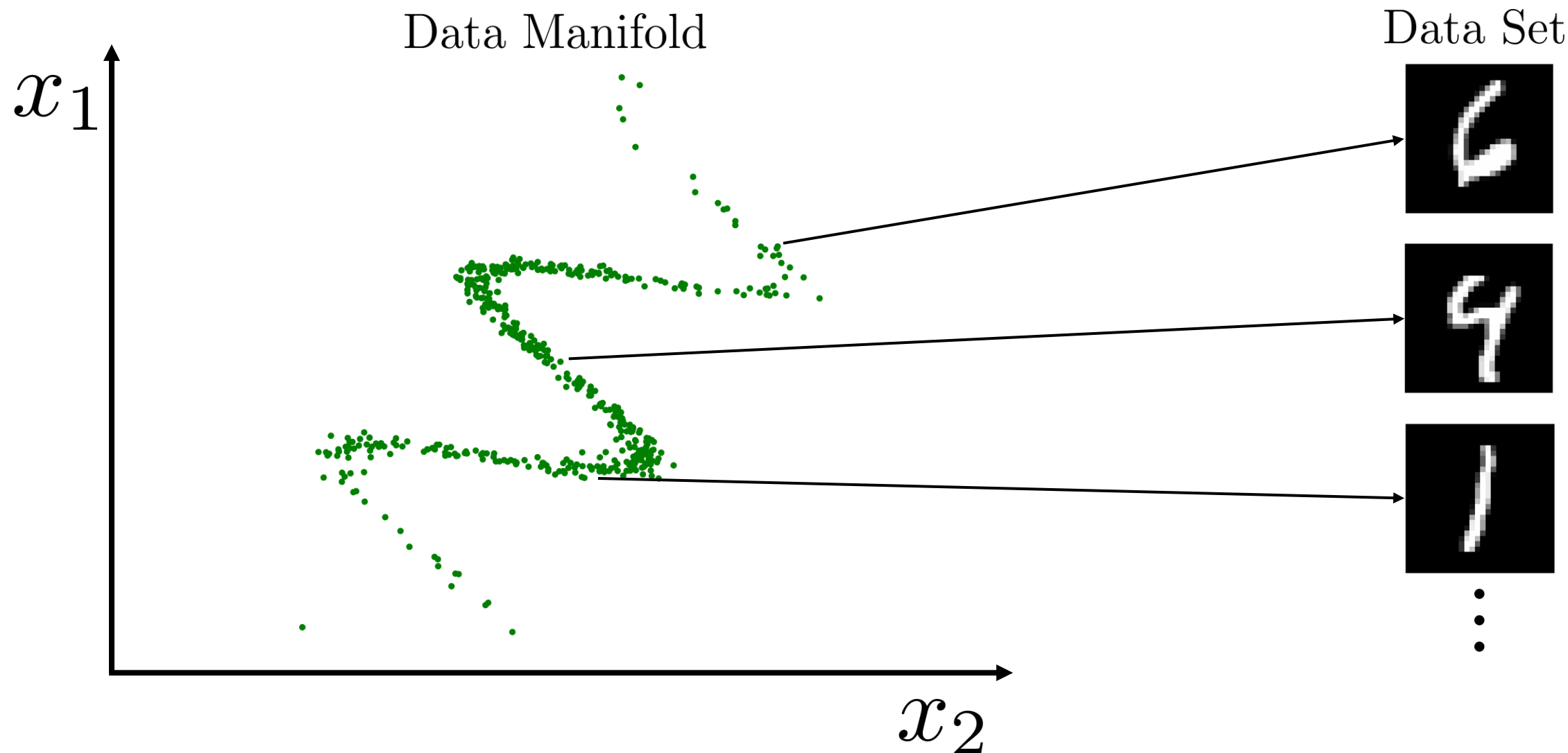


⋮

But natural images lie on a manifold ...

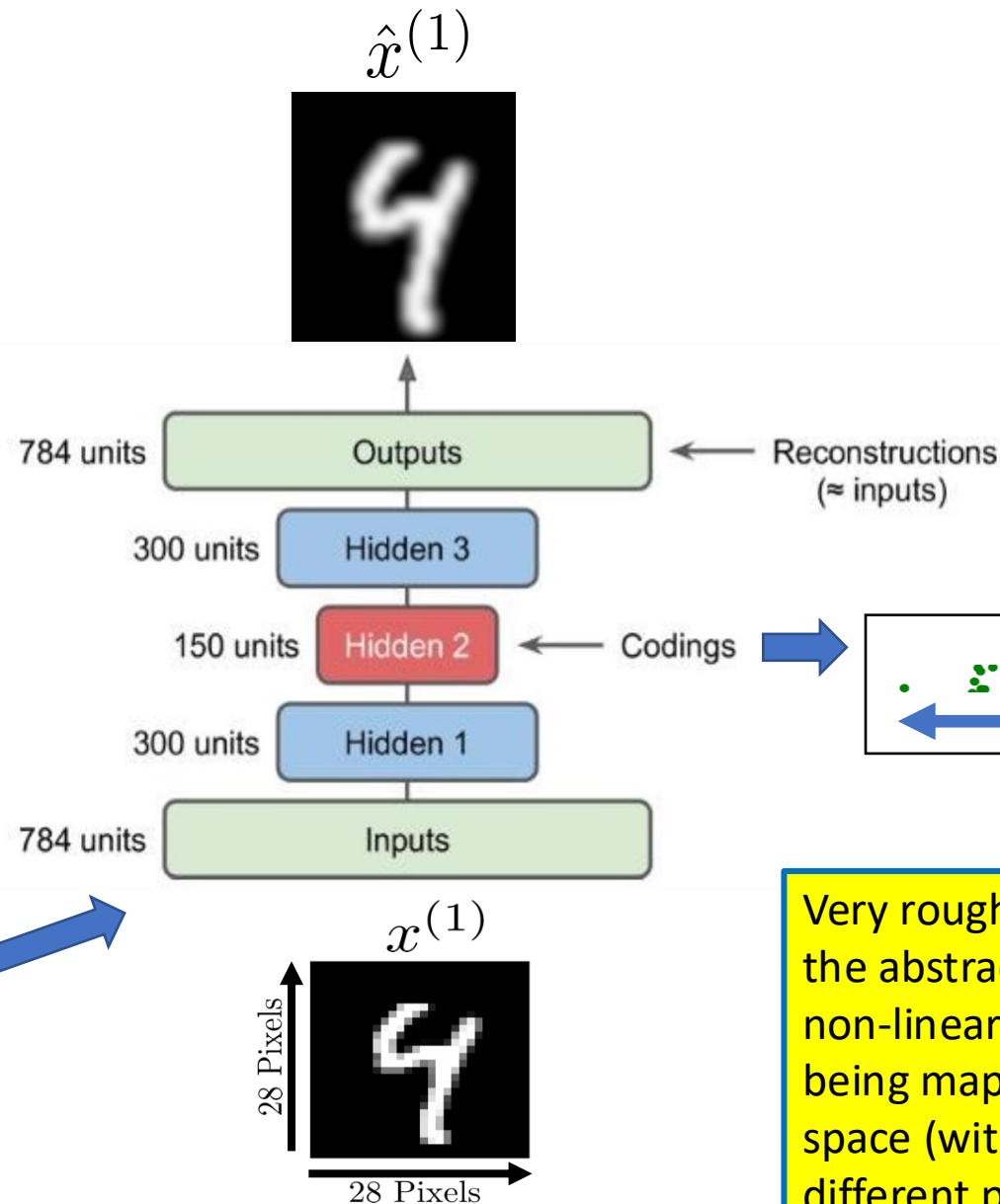
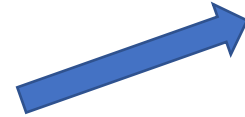
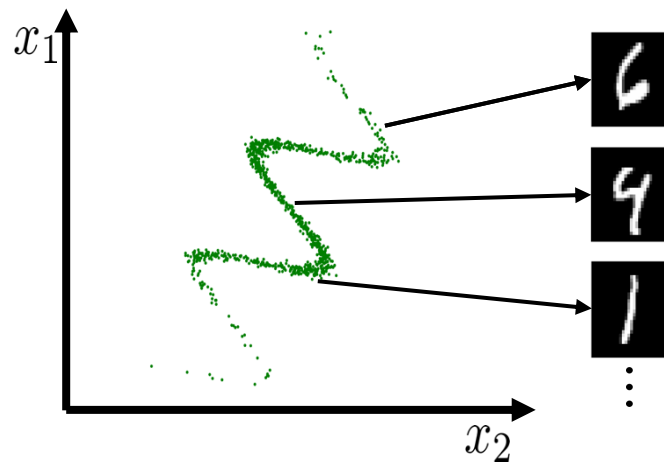


University
of Glasgow



Non-linear autoencoders

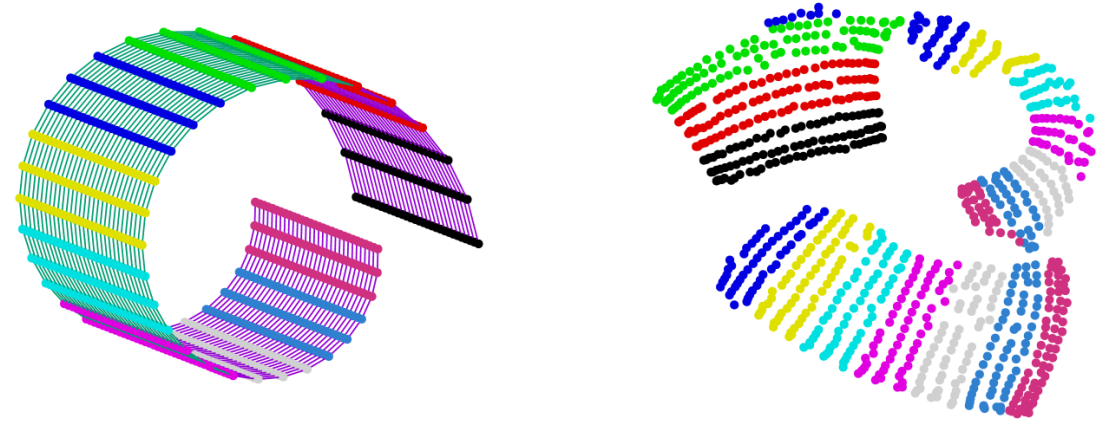
The MNIST digits are 784 dimensional and they are being mapped to 150 dimensional latent space for reasonable reconstruction of the images.



Very rough **schematic** showing the abstract idea that a 1D non-linear manifold in 2D space is being mapped onto a 1D latent space (with different digits at different positions).

2D manifold in 3D space ...

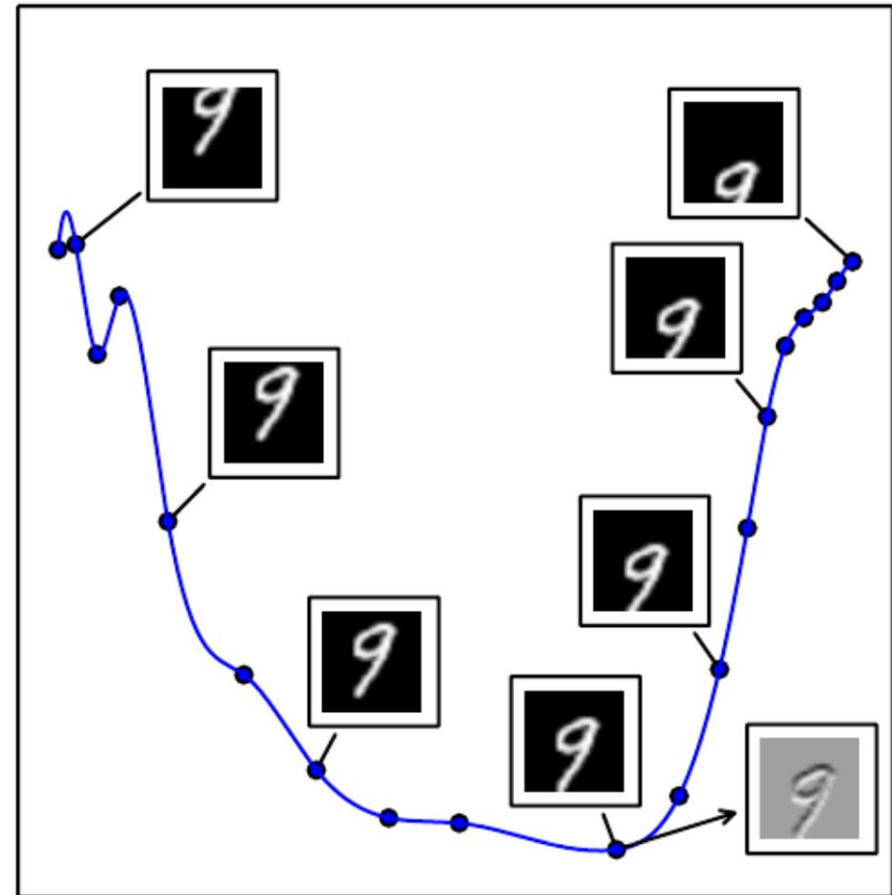
- A non-linear autoencoders can do something similar to non-linear dimensionality reduction approaches such as t-SNE.
- 2D swiss roll manifold embedded within 3D space being “flattened”.



t-SNE unrolling of the swiss roll (with one noise dimension)

Manifold Learning

- This example is translating the digit “9” of the MNIST dataset.
- All of these shifted digit 9s are valid data samples that live in a very high dimensional space (28 x 28).
- These ones have been projected to 2D using t-SNE and we can see the 1D manifold these samples lie upon.
- A tangent is shown going from one sample to another and the difference in images (grey = pixels values stay same, white = pixel value increases, black = pixel value decreases).



Manifolds have much higher dimensions for their tangent planes

- An image with different head orientations are all suitable image samples that lie on the manifold of valid face images.
- (Again, this only shows 2D manifold example in 3D whereas the real manifolds are much higher dimensional. For instance, when we talk, there are lots of “degrees of freedom” as dimensions in the manifold.)

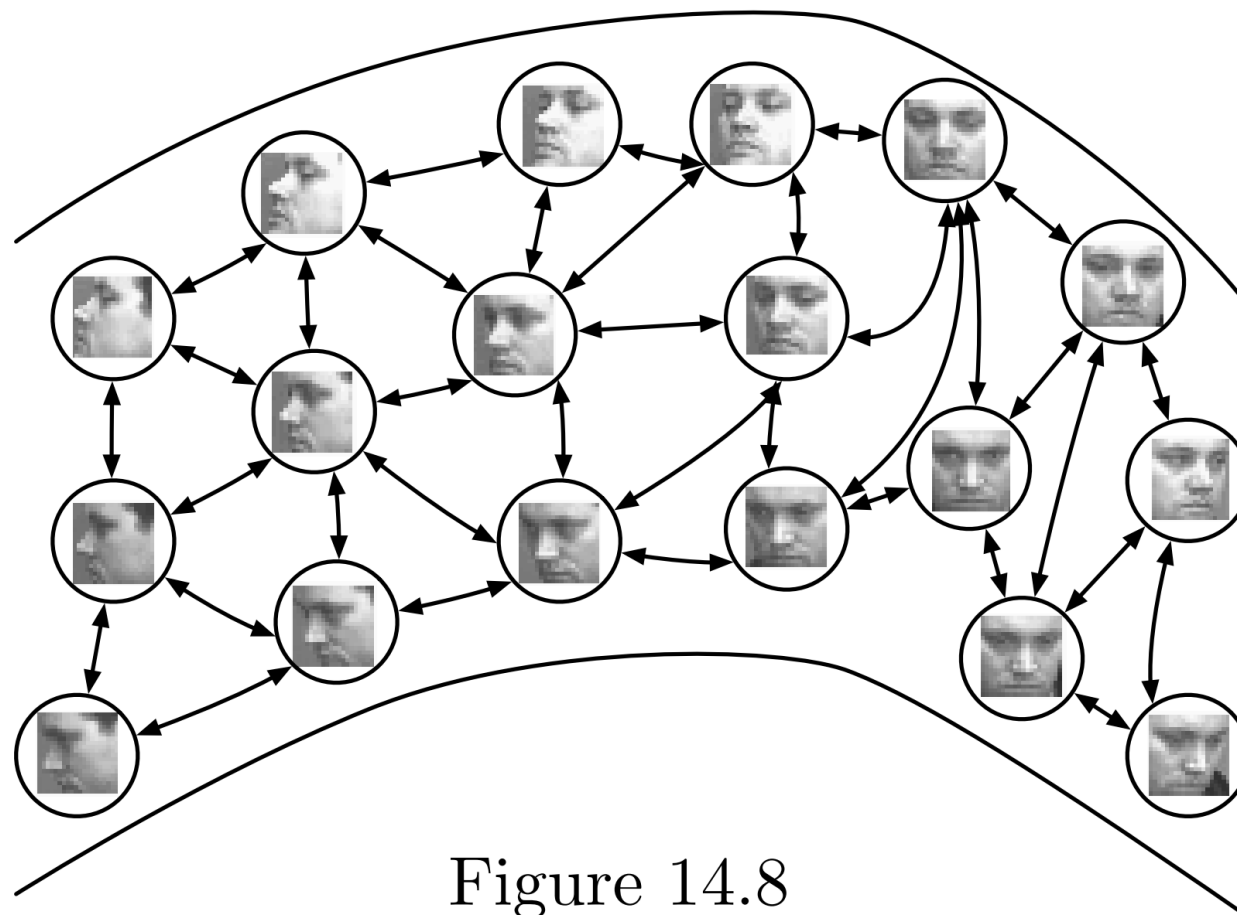
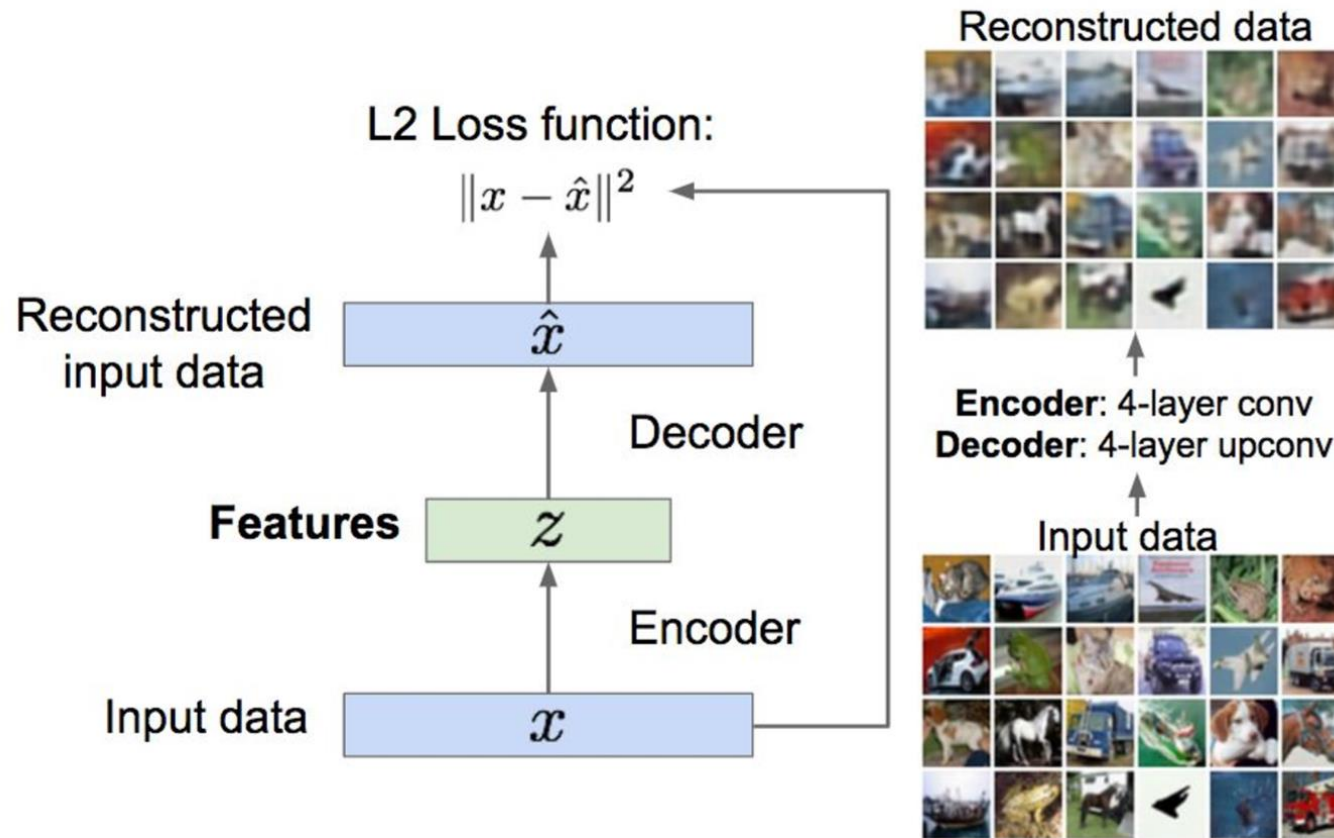


Figure 14.8

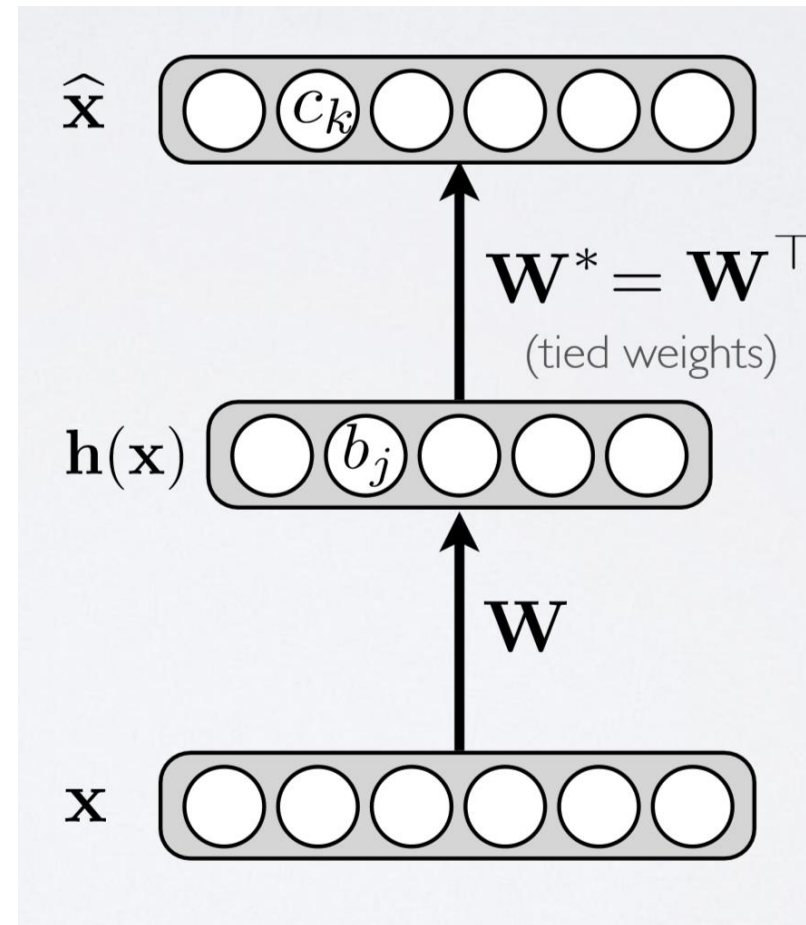
Convolutional autoencoder doing unsupervised feature learning on images



Autoencoders
(Feature learning)

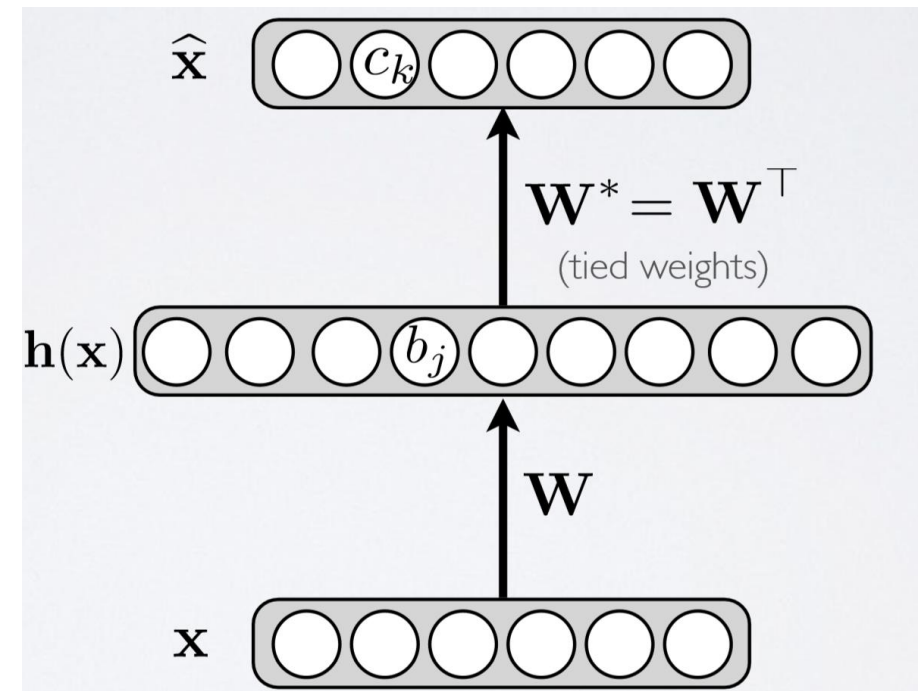
Undercomplete autoencoders

- So far we have been looking at **undercomplete autoencoders** where the latent space h has a lower dimension than x .
- This smaller dimension stops it **trivially copying** x into the latent space and reconstructing it by copying it out again !
- We hope it will learn in the latent space the salient features of the data – thus allowing (lossy) compression of the data or feature extraction.
- But if the encoder/decoder allows to be far too complex ... it could just learn to encode each data sample $x^{(i)}$ with the integer value i in just a 1 dimensional latent space which would not be useful (or generalize to any other data !)



Overcomplete autoencoders

- This is another type of autoencoder where the latent space h has a higher dimension than x .
- But then there is a real danger that the autoencoder may just learn to **trivially copy** x into the latent space and reconstruct it by copying it out again !
- And why would we want to use an autoencoder like this ?
- It allows us to add constraints so that the encoding has particular desirable properties that we would like to see (for instance sparsity).
- Essentially we need to add **regularization** when using an overcomplete autoencoder. This will hopefully also stop the trivial solution of just copying x to the higher dimensional latent space.



Regularized Autoencoders

- Sparse encoders
- Denoising autoencoders
- Autoencoders with drop-out
- Contractive autoencoders

Sparse Autoencoders

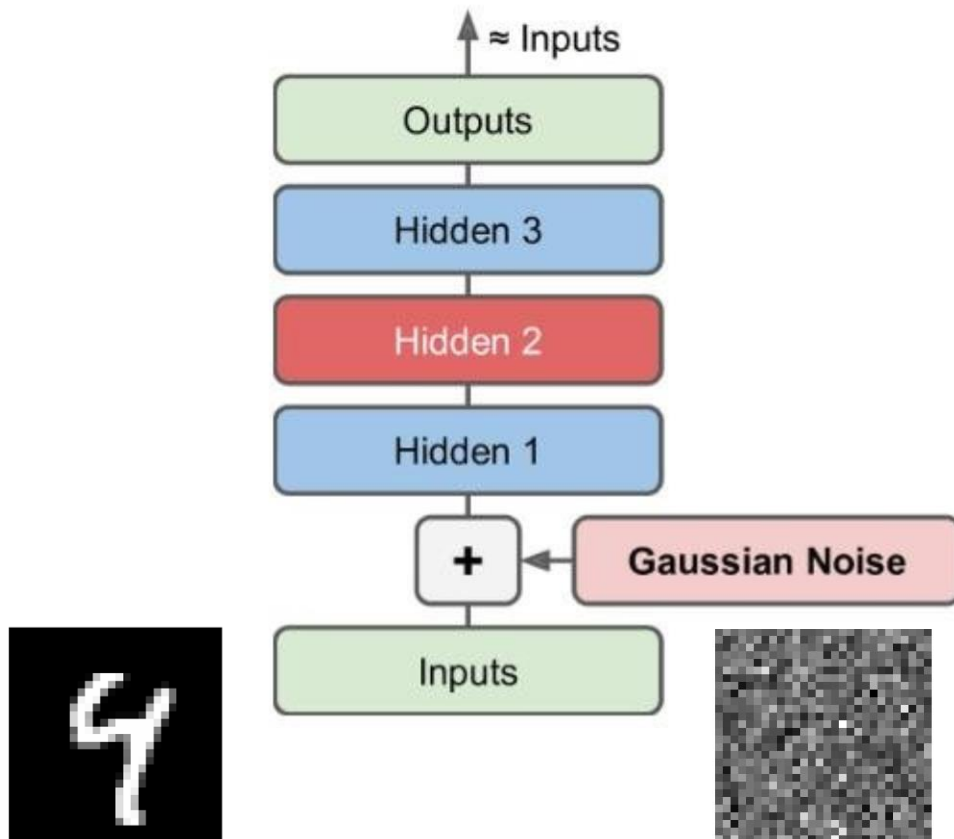
Limit capacity of autoencoder by adding a sparsity penalty term $\Omega(h)$ to the cost function penalizing the code layer h for being larger

$$L(x, g(f(x))) + \Omega(h)$$

where $g(h)$ is the decoder output and typically $h = f(x)$, the encoder input.

- Often L1 regularization is added as the penalty term to encourage the latent vector h to be sparse.
- Sparsity encourages the latent space to have unique statistical features of the training samples so that it can reconstruct them.
- Sparse encoders are often used to learn features for another task such as classification (thus unsupervised pre-training is being used for supervised networks)

Denoising Autoencoders



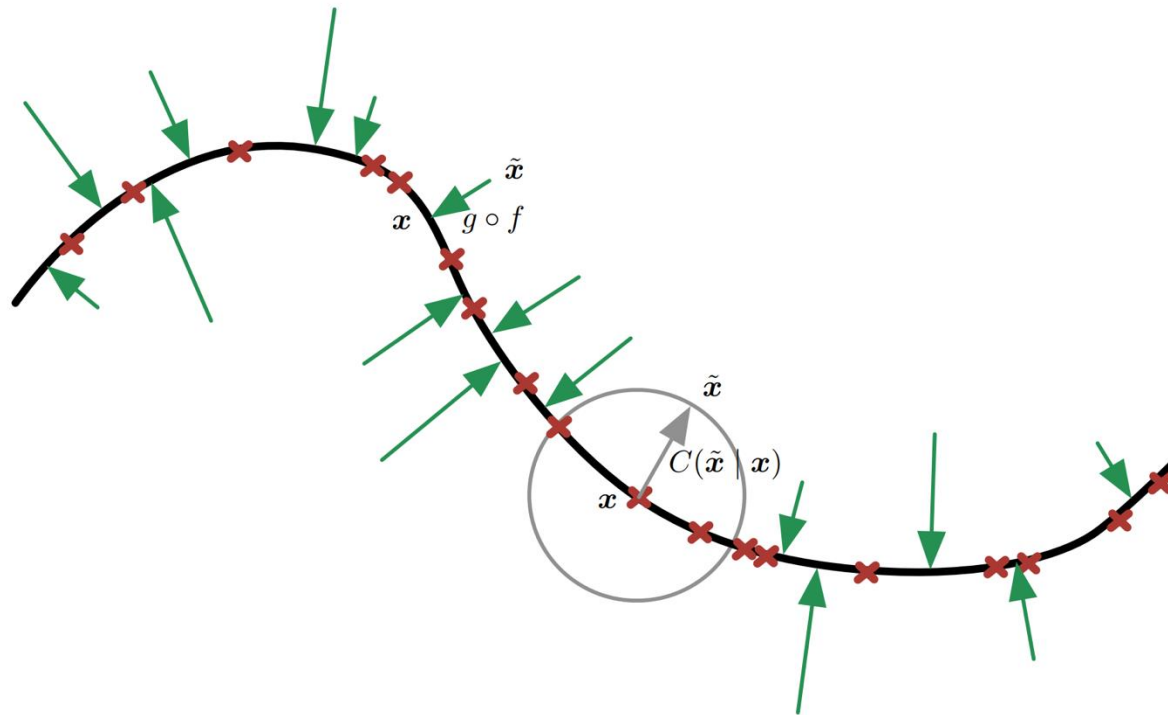
A denoising autoencoder adds Gaussian noise to the input image ... and then the autoencoder has to reconstruct the original image.

Alternatively drop-out can be used to exclude parts of the input image ... and then the autoencoder has to reconstruct the original image.

Both approaches stop the autoencoder just learning a trivial identity function.

(Although the denoising autoencoder is also useful to take noise out of images ...)

Denoising autoencoders is learning a manifold of the data



Again, we take the view that the samples are living on a much lower dimensional manifold (or embedding) within a high dimensional space.

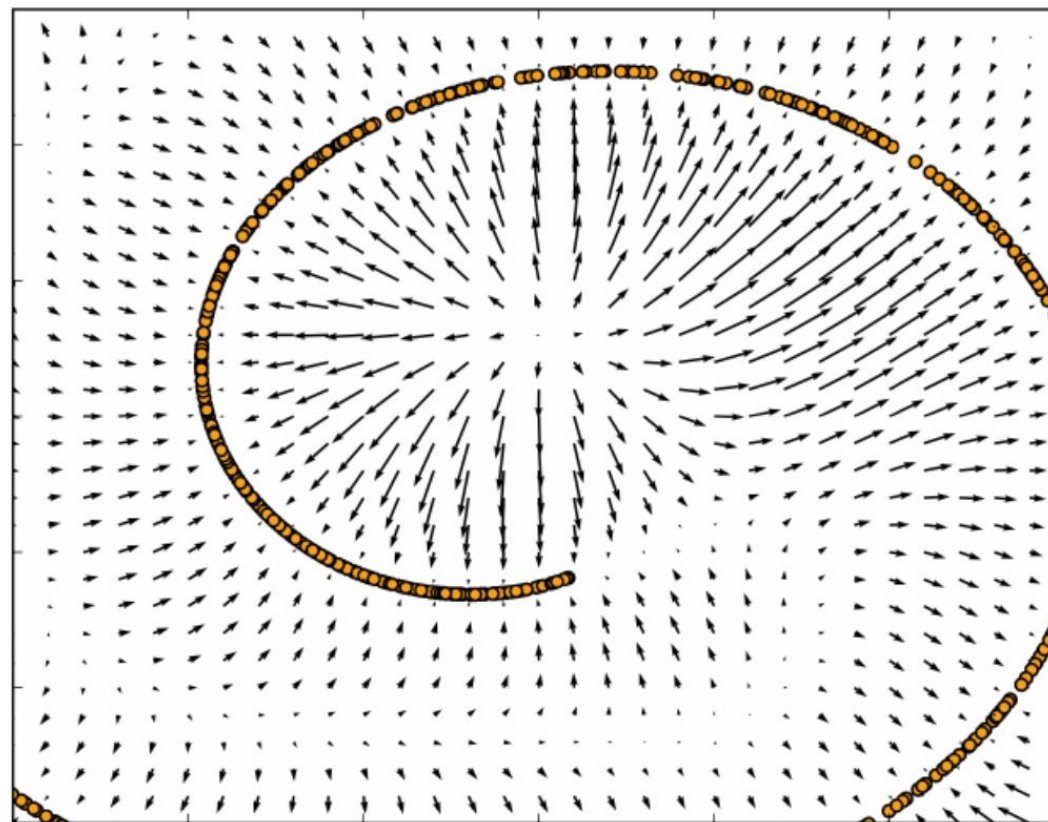
Adding Gaussian noise is pushing them off this manifold in a random direction (grey circle in diagram).

This is more likely to be away from the manifold since manifold has a much lower dimension than the total space.

Denoising autoencoder is learning to move points outside the manifold (i.e. with random noise) back onto the manifold (green arrows in diagram).

Vector field learned by autoencoder

- Simple example with data points lying on a curved 1D manifold in 2D space. (Real examples live in much higher dimensional spaces.)
- The denoising autoencoder learns the vector field of how to move points outside the manifold back onto the manifold.
- It learns the higher probability regions of the space where actual data samples reside. (You would hope that we could generate new examples of the data by sampling these high probability regions of space)



(Example diagram from: Alain and Bengio, 2013)

Contractive Autoencoders

The contractive autoencoder (Rifai *et al.*, 2011a,b) introduces an explicit regularizer on the code $\mathbf{h} = f(\mathbf{x})$, encouraging the derivatives of f to be as small as possible:

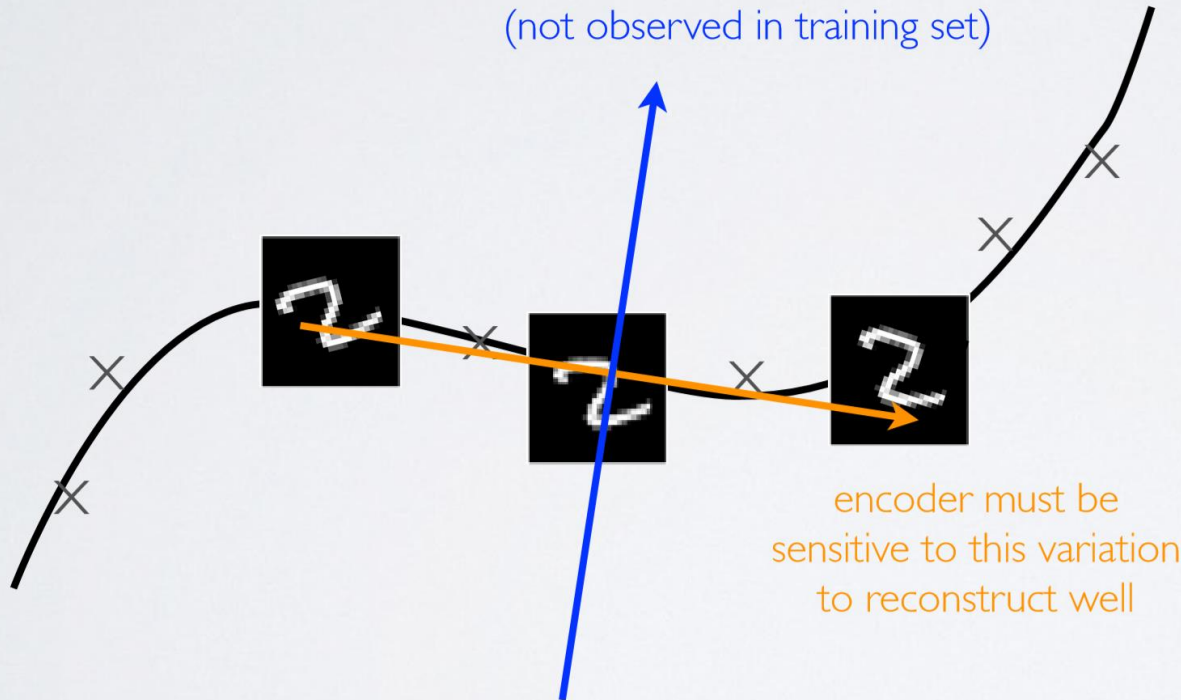
$$\Omega(\mathbf{h}) = \lambda \left\| \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2. \quad (14.18)$$

The penalty $\Omega(\mathbf{h})$ is the squared Frobenius norm (sum of squared elements) of the Jacobian matrix of partial derivatives associated with the encoder function.

- So this is a similar approach to the sparse autoencoder adding an L1-norm to the loss function.
- But it is adding a norm which is trying to make the gradients of the encoder (with respect to input values) as small as possible.
- This encourages the latent value to only change if absolutely needed (i.e. along the surface of the manifold itself).

- Illustration:

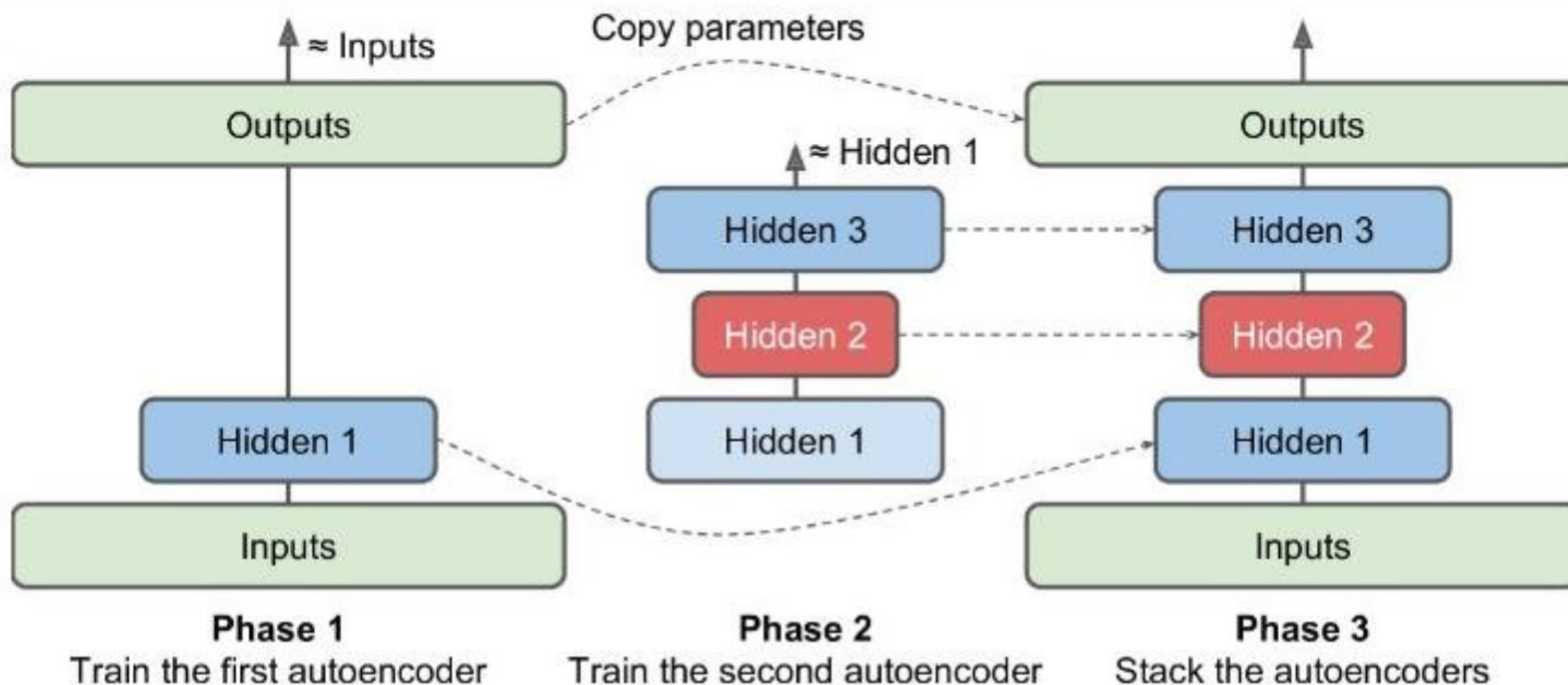
encoder doesn't need to be
sensitive to this variation
(not observed in training set)



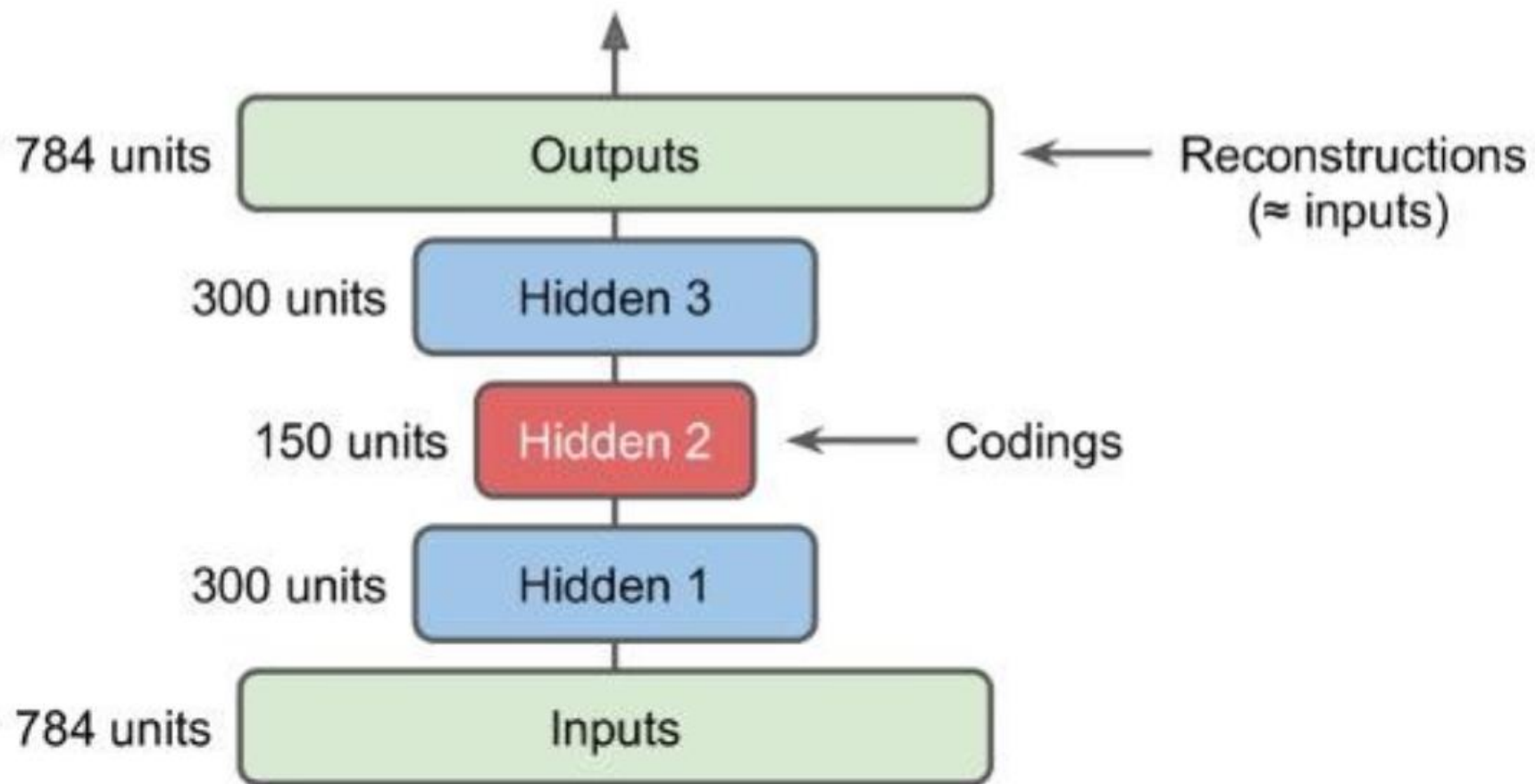
- It can be shown that actually the contractive autoencoder is equivalent to the denoising autoencoder as the Gaussian noise tends to zero !
- **Denoising Autoencoder:**
Simpler to implement
- **Contractive Autoencoder:**
deterministic, can be more stable, but harder to implement.

"Contractive Auto-Encoders: Explicit Invariance During Feature Extraction. Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot et Yoshua Bengio, 2011.

Special training for contractive AE - Training one autoencoder at a time



Stacked autoencoders



Summary

- Autoencoders can carry out a number of unsupervised learning tasks.
- They typically learn a latent space (an embedding for the training data) within a much larger input space.
- They can be used for:
 - Dimensionality reduction and visualization.
 - Unsupervised feature extraction for supervised networks (often called unsupervised pre-training).
 - In a similar manner - getting embedding vectors for data samples (compare with word embeddings).
 - Denoising data.
 - Analyzing the manifold that data lies within (contractive autoencoder with tangents to the manifold).
- They can be:
 - Undercomplete – latent space smaller than the input space
 - Overcomplete – latent space larger than the input space
- For overcomplete autoencoders, regularization needs to be applied:
 - Adding L1 regularization to the loss function to encourage sparse latent vectors.
 - Denoising autoencoders and autoencoders that use drop-out
 - Contractive autoencoders where a sum of squares of Jacobian gradients is added to the loss function.
- Each gives autoencoders with particular desirable properties.

Background Reading: Goodfellow Chapter 14.